## Practical No. 1: Mini Generative AI – Next Word Prediction

### Aim

To understand the basic concept of Generative Artificial Intelligence by developing a simple next-word prediction model using Python.

### Theory

Generative Artificial Intelligence (Generative AI) is a branch of AI that creates new content such as text, images, audio, and code by learning patterns from existing data. One of the simplest text generation techniques is **next-word prediction**, where the system predicts the most probable next word based on previously learned word sequences. This concept is the basic building block of modern Large Language Models (LLMs) like ChatGPT.

### Algorithm

1. Start the program.
2. Create a list of sample sentences.
3. Split each sentence into individual words.
4. Store every word along with its next word in a dictionary.
5. Display the learned model.
6. Accept a starting word from the user.
7. Randomly select the next word from the learned dictionary.
8. Repeat the process until no next word is available or the desired number of words is generated.
9. Display the generated text.
10. Stop.

### Python Program

```
import random

# Training sentences
sentences = [
    "I love data science",
    "I love machine learning",
    "data science is amazing",
    "machine learning is powerful",
    "I enjoy learning python"
]

# Dictionary to store word relationships
model = {}

# Build the model
for sentence in sentences:
    words = sentence.split()
```

```
    for i in range(len(words) - 1):
        current_word = words[i]
        next_word = words[i + 1]

        if current_word not in model:
            model[current_word] = []

        model[current_word].append(next_word)

print("Learned Model")
print(model)

# User input
word = input("\nEnter starting word: ")

generated_text = word

# Generate text
for i in range(5):

    if word in model:
        word = random.choice(model[word])
        generated_text = generated_text + " " + word
    else:
        break

print("\nGenerated Text:")
print(generated_text)
```

**Sample Output**

```
Learned Model
{'I': ['love', 'love', 'enjoy'], 'love': ['data', 'machine'], 'data': ['science', 'science'], 'machine':

Enter starting word: love

Generated Text:
love data science is amazing
```

**Conclusion**

The practical demonstrated the implementation of a simple next-word prediction model using Python. It helped in understanding how Generative AI learns patterns from text and generates meaningful sentences by predicting one word at a time. This concept forms the foundation of modern text generation models.

## Practical No. 2: Text Generation and Text Summarization using Generative AI

### Aim

To understand the concepts of text generation and text summarization using Python.

### Theory

Generative AI can create new text as well as summarize lengthy content into a shorter form. Text generation involves producing meaningful sentences based on given input, while text summarization extracts the most important information from a paragraph. These techniques are widely used in chatbots, content creation, search engines, virtual assistants, and document analysis systems.

### Part A: Text Generation

### Objective

To generate text based on user input using predefined responses.

### Algorithm

1. Start the program.
2. Create a dictionary containing keywords and corresponding generated text.
3. Accept input from the user.
4. Check whether the keyword exists in the dictionary.
5. Display the generated text.
6. Stop the program.

### Python Program

```
responses = {
    "ai": "Artificial Intelligence is transforming the world.",
    "python": "Python is a popular programming language for AI.",
    "data": "Data is the foundation of machine learning."
}

word = input("Enter a keyword (ai/python/data): ").lower()

if word in responses:
    print("\nGenerated Text:")
    print(responses[word])
else:
    print("No text available for the given keyword.")
```

**Output**

```
responses = {
    "ai": "Artificial Intelligence is transforming the world.",
    "python": "Python is a popular programming language for AI.",
    "data": "Data is the foundation of machine learning."
}

word = input("Enter a keyword (ai/python/data): ").lower()

if word in responses:
    print("\nGenerated Text:")
    print(responses[word])
else:
    print("No text available for the given keyword.")
```

```
Enter a keyword (ai/python/data): python

Generated Text:
Python is a popular programming language for AI.
```

**Part B:** Text Summarization using Hugging Face Transformer

**Objective**

To summarize a paragraph by extracting important sentences.

**Algorithm**

1. Start the program.
2. Import the pipeline function from the transformers library.
3. Load the pre-trained summarization pipeline.
4. Store the input paragraph in a variable.
5. Generate the summary by specifying the maximum and minimum summary length.
6. Display the generated summary.
7. Stop the program.

**Python Program**

Step 1: Uninstall the existing packages

```
!pip uninstall -y transformers torch torchvision torchaudio
```

Step 2: Install compatible versions

```
!pip install -q torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0
!pip install -q transformers==4.52.4 sentencepiece accelerate
```

Step 3: Restart the Notebook Session

Go to **Run → Restart Session** (or restart the kernel).

Step 4: Run Your Program

```
from transformers import pipeline
# Load summarization pipeline
summarizer = pipeline(
    task="summarization",
    model="facebook/bart-large-cnn"
)

# Input paragraph
text = """
Artificial Intelligence is transforming every industry by automating tasks,
improving decision-making, and generating human-like content.
Generative AI is a branch of AI capable of creating text, images,
music, videos, and even software code.
"""

# Generate summary
summary = summarizer(text, max_length=40, min_length=15)

# Display original text
print("Original Text:\n")
print(text)

# Display summary
print("\nGenerated Summary:\n")
print(summary[0]["summary_text"])
```

**Output**

```
Generated Summary:

Generative AI is a branch of AI capable of creating text, images,music, videos, and even software code.
```

**Conclusion**

The practical demonstrated how a pre-trained **BART Transformer** model can summarize lengthy text into a concise and meaningful form. It provided hands-on experience in using Generative AI for Natural Language Processing (NLP) tasks and highlighted the practical applications of AI-powered text summarization in education, business, and content analysis.

## Practical No. 3: AI Tutor Chatbot using Google Gemini API

### Aim

To develop an AI Tutor Chatbot using the Google Gemini API that interacts with users and answers programming-related questions in a conversational manner.

### Theory

A chatbot is a conversational AI application that communicates with users using Natural Language Processing (NLP). Modern chatbots are powered by **Large Language Models (LLMs)**, which understand user queries and generate human-like responses. In this practical, the **Google Gemini API** is used to create an AI Tutor that explains coding concepts to beginners. A **system instruction** is provided to define the chatbot's personality, making it friendly, encouraging, and suitable for learning.

### Algorithm

1. Start the program.
2. Import the required google.genai library.
3. Initialize the Gemini client using the API key.
4. Create a chat session using the **Gemini 2.5 Flash** model.
5. Set the system instruction to define the chatbot's role as an AI tutor.
6. Display a welcome message.
7. Accept user input in a loop.
8. Send the user's message to the Gemini model.
9. Display the AI-generated response.
10. Repeat until the user enters **exit**.
11. Stop the program.

### Python Program

```
import os
from google import genai

# Initialize Gemini client
client = genai.Client(api_key="AQ.Ab8RN6JkmZk7st62RATnSJpe-0NqOvT2JBmN-YSJ0eWorqxnVw")

# Create AI Tutor Chat
chat = client.chats.create(
    model="gemini-3.5-flash",
    config=genai.types.GenerateContentConfig(
        system_instruction="You are a brilliant AI tutor explaining coding to beginners. Be encouraging and use emojis."
    )
)

print("AI Tutor: Hello! I'm your GenAI tutor. Ask me anything!")

while True:
```

```
user_input = input("You: ")

if user_input.lower() == "exit":
    print("AI Tutor: Thank you! Happy Learning 😊")
    break

response = chat.send_message(user_input)

print("AI Tutor:", response.text)
print()
```

**Output**

```
AI Tutor: Hello! I'm your GenAI tutor. Ask me anything!
You: what is gen ai
AI Tutor: Hello there, future tech superstar! 🌟 That is an absolute **excellent** question to ask.

Let's dive into the exciting world of **Gen AI** (which is short for **Generative Artificial Intelligence**

---

### What is Gen AI? 🤔

In the simplest terms, **Generative AI** is a type of artificial intelligence that can **create brand-new t

Before Gen AI, older AI systems were mostly like "judges" or "detectives." They could look at a photo and s

But **Generative AI** is like an **artist, writer, and programmer** all rolled into one! Instead of just re

---

### What can Gen AI create? 🌟

It's not just one thing! Gen AI can create all sorts of cool stuff:

1. **Text 📝:** It can write essays, poems, emails, or even chat with you (just like I am doing right now!)
2. **Images 🎨:** You can type *"a cute golden retriever puppy wearing a space suit on Mars"* and it will c
3. **Code 💻:** (My personal favorite!) It can write computer code in languages like Python, JavaScript, or
4. **Music & Video 🎵🎥:** It can compose songs or make short video clips from just a text description.

---
```

**Conclusion**

The practical demonstrated the development of an AI Tutor Chatbot using the Google Gemini API. The chatbot interacted with users in natural language, answered programming-related questions, and provided beginner-friendly explanations. This practical helped in understanding the implementation of conversational AI using Large Language Models (LLMs).

## Practical No. 4: Introduction to Generative AI using Gemini API

### Aim

To understand the basic concepts of Generative AI and perform different text-generation tasks using the Gemini API and Python.

### Theory

Generative AI is a branch of Artificial Intelligence that can generate new content such as text, images, audio, code, and other forms of information. In this practical, the Gemini API is used with Python to perform various text-generation tasks through suitable prompts.

### Software/Requirements

- Python
- Google Colab / Jupyter Notebook
- Gemini API
- google-generativeai Python library

### Sub-Part 1: Basic Text Generation

### Problem Statement

Write a Python program to use Generative AI to explain Artificial Intelligence in simple words.

### Program

```
!pip -q install google-generativeai


import google.generativeai as genai

API_KEY = "AQ.Ab8RN6JkmZk7st62RATnSJpe-0NqOvT2JBmN-YSJ0eWorqxnVw"

genai.configure(api_key=API_KEY)

model = genai.GenerativeModel("gemini-3.5-flash")

response = model.generate_content("Explain Artificial Intelligence in simple words.")

print(response.text)
```

**Output**

```
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
At its simplest, **Artificial Intelligence (AI)** is when computers and machines are programmed to "think," learn, and make de

Normally, computers can only do exactly what we tell them to do. But with AI, computers can look at information, learn from it

Here is an easy way to understand how it works:

### 1. How AI learns (The "Dog" Example)
Imagine you want to teach a child what a dog is. You don't give them a list of rules like, *"A dog has four legs, fur, and a ta

AI works the exact same way:
* We show a computer **millions of pictures** of dogs.
```

**Sub-Part 2: AI-Based Story Generation**

**Problem Statement**

Write a Python program to generate a short story on a topic entered by the user using Generative AI.

**Program**

```
topic = input("Enter topic: ")

prompt = f"Write a short story on {topic}"

response = model.generate_content(prompt)

print(response.text)
```

**Output**

Enter topic: Machine Learning

```
Enter topic: machine learning
The servers in Basement Lab 4 hummed like a colony of digital cicadas. For six months, Dr. Elena Vance had liv

Aura was a neural network designed for a singular, monumental task: to translate the untranslatable.

Elena's dataset consisted of thousands of ancient, clay tablets from a lost Bronze Age civilization, the *Kiri

Elena was trying a different approach. She wasn't teaching Aura grammar rules. Instead, she was feeding the ma

"Epoch 1," Elena whispered, clicking the run script.

At first, Aura was blind. Her "weights"—the millions of mathematical connections mimicking biological synapses

Elena watched the monitor. "Gradient descent," she murmured.

With every mistake, Aura adjusted her parameters, taking a tiny step down a steep mathematical mountain toward

By Epoch 1,000, Aura had discovered nouns.
```

**Sub-Part 3: Question Answering using Generative AI**

**Problem Statement**

Write a Python program that accepts a question from the user and generates an answer using Generative AI.

**Program**

```
question = input("Ask anything: ")

response = model.generate_content(question)

print(response.text)
```

**Output**

```
Ask anything: how does PYTHON WORKS?
To understand how Python works, it helps to look at it from two angles: **how your code is ex

Here is a step-by-step breakdown of how Python works, from the moment you write a line of cod

---

### Part 1: The 3-Step Execution Process

When you run a Python program (like `hello.py`), Python does not immediately run the code on

```text
[ Source Code (.py) ]
       |
       ▼ (Step 1: Compilation)
[ Bytecode (.pyc) ]
```

**Sub-Part 4: Text Translation using Generative AI**

**Problem Statement**

Write a Python program to translate a given English sentence into Hindi using Generative AI.

**Program**

```
text = "Artificial Intelligence is the future."

prompt = f"Translate into Hindi:\n{text}"

response = model.generate_content(prompt)

print(response.text)
```

**Sample Output**

```
Here is the translation of "Artificial Intelligence is the future" in Hindi:

**कृत्रिम बुद्धिमत्ता ही भविष्य है।**
(Pronounced: *Kritrim buddhimatta hi bhavishya hai*)

Alternatively, in modern/common Hindi, you can also say:
**आर्टिफिशियल इंटेलिजेंस ही भविष्य है।**
```
```

**Sub-Part 5: Professional Content Generation**

**Problem Statement**

Write a Python program to generate a professional leave application for two days using Generative AI.

**Program**

```
prompt = """

Write a professional leave application for 2 days.

"""

response = model.generate_content(prompt)

print(response.text)
```

**Sample Output**

```
Here are three professional templates you can use for a 2-day leave application. Choose the one that best fits your situation (General/Pers

### Option 1: General/Personal Leave (Best for planned personal days)

**Subject:** Leave Application: [Your Name] – [Start Date] to [End Date]

Dear [Manager's Name],

I am writing to formally request two days of leave, starting from [Start Date] to [End Date]. I need to attend to some personal matters tha

Before my departure, I will ensure that all my pending tasks are up to date. I have also requested [Colleague's Name] to support any urgent
```

**Sub-Part 6: Multiple-Choice Question Generation**

**Problem Statement**

Write a Python program to generate 10 multiple-choice questions with answers on a given topic using Generative AI.

**Program**

```
topic = "Machine Learning"

prompt = f"""

Generate 10 multiple-choice questions on {topic}

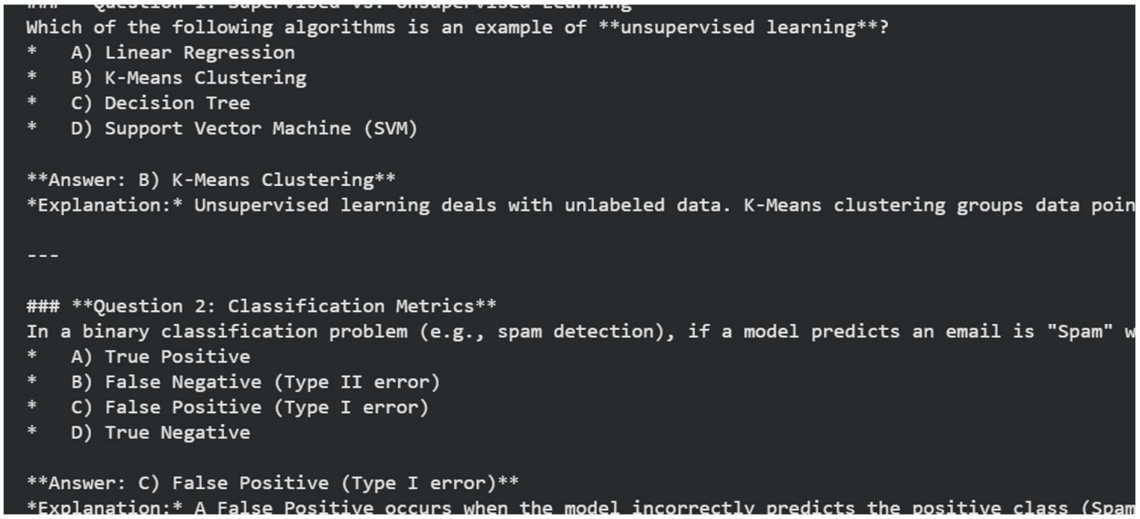
with answers.

"""

response = model.generate_content(prompt)

print(response.text)
```

## Output



```
### **Question 1: Supervised vs. Unsupervised Learning**
Which of the following algorithms is an example of **unsupervised learning**?
*   A) Linear Regression
*   B) K-Means Clustering
*   C) Decision Tree
*   D) Support Vector Machine (SVM)

**Answer: B) K-Means Clustering**
*Explanation:* Unsupervised learning deals with unlabeled data. K-Means clustering groups data poin

---

### **Question 2: Classification Metrics**
In a binary classification problem (e.g., spam detection), if a model predicts an email is "Spam" w
*   A) True Positive
*   B) False Negative (Type II error)
*   C) False Positive (Type I error)
*   D) True Negative

**Answer: C) False Positive (Type I error)**
*Explanation:* A False Positive occurs when the model incorrectly predicts the positive class (Spam
```

## Conclusion

The practical demonstrated different applications of Generative AI using Python and the Gemini API, including text generation, story generation, question answering, translation, professional content creation, and MCQ generation. The next-word prediction sub-part further demonstrated the basic principle of text generation by learning word-to-word relationships from sample sentences.

## Practical No. 5: Text Generation and Prompt-Based Response Generation using Generative AI

### Aim

To understand and implement different text-generation applications using pre-trained Generative AI models and Python.

### Theory

Generative AI models are capable of creating new text based on a given prompt. Pre-trained language models can be used to complete sentences, generate responses, and perform various natural language tasks.

In this practical, two approaches to Generative AI are demonstrated. The first sub-part uses the **GPT-2 pre-trained model** through the Hugging Face Transformers library for local text generation. The second sub-part uses the **Gemini API** to generate a customer-service response based on a predefined company policy and customer message.

### Software/Requirements

- Python
- Google Colab / Jupyter Notebook
- Hugging Face Transformers
- PyTorch
- Gemini API
- Internet connection

### General Algorithm

1. Start the program.
2. Install and import the required libraries.
3. Select the required Generative AI model.
4. Provide an appropriate prompt or input.
5. Generate text using the selected model.
6. Display the generated response.
7. Stop.

### Sub-Part 1: Local Text Generation using GPT-2

### Problem Statement

Write a Python program to generate and complete text using the pre-trained GPT-2 language model available through the Hugging Face Transformers library.

**Theory**

GPT-2 is a pre-trained language model developed for natural language generation. It predicts the next words based on the words already provided in a prompt.

The Hugging Face pipeline() function provides a simple way to use pre-trained models for tasks such as text generation. In this program, GPT-2 generates text based on a starting sentence.

**Algorithm**

1. Install the transformers and torch libraries.
2. Import the pipeline function from Transformers.
3. Create a text-generation pipeline using GPT-2.
4. Provide a starting sentence as the prompt.
5. Generate text using the model.
6. Set the maximum length of the generated text.
7. Display the generated text.
8. Stop.

**Python Program**

```
!pip install transformers torch

from transformers import pipeline

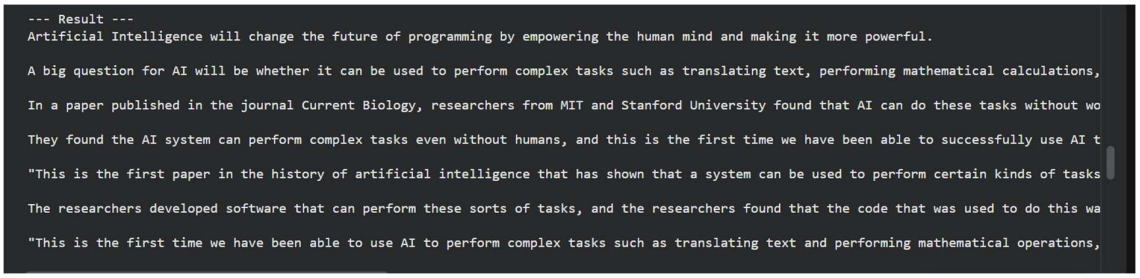
def generate_local_text(prompt_text: str):
    print("Loading model... (This may take a moment on the first run)")


    # Initialize a text-generation pipeline using standard open-source gpt2
    generator = pipeline('text-generation', model='gpt2')


    print("Generating response...")
    # Generate text based on the prompt
    outputs = generator(
        prompt_text,
        max_length=60,
        num_return_sequences=1,
        pad_token_id=50256  # Prevents warning output
```

```
    )
    return outputs[0]['generated_text']
if __name__ == "__main__":
    # Give the model a starting sentence to complete
    sample_prompt = "Artificial Intelligence will change the future of programming by"
    result = generate_local_text(sample_prompt)
    print("\n--- Result ---")
    print(result)
```

**Output**




**Note:** Since GPT-2 generates text probabilistically, the exact output may be different each time.

**Sub-Part 2: Customer Service Response Generation using Gemini**

**Problem Statement**

Write a Python program to generate a polite customer-service email response using Generative AI based on a predefined refund policy and customer message.

**Theory**

Generative AI can generate context-specific responses by combining **instructions, rules, and user input** into a single prompt. This technique is commonly used in customer support applications.

In this program, a refund policy and a customer email are provided to the Gemini model. The model uses the given policy to generate an appropriate and polite response.

**Algorithm**

1. Import the Gemini API library.
2. Create a Gemini client using the API key.
3. Define the company refund policy.
4. Define the customer email.
5. Combine the policy and customer message into a prompt.

6. Send the prompt to the Gemini model.
7. Generate the response.
8. Display the drafted email reply.
9. Stop.

**Python Program**

```
import os

from google import genai


client = genai.Client(api_key="AQ.Ab8RN6JkmZk7st62RATnSJpe-0NqOvT2JBmN-YSJ0eWorqxnVw")

# 1. Provide your custom rules

refund_policy = "Policy: We only allow refunds if the item is broken and reported within 7 days."

# 2. Simulate an incoming customer message

customer_email = "Hi, I bought a shirt 2 days ago but the zipper is completely broken. Can I get my money back?"

# 3. Combine rules and the message into a prompt

prompt = f"""

Use our company policy to answer the customer email politely.

{refund_policy}

Customer Email: {customer_email}

"""

# 4. Generate the reply

response = client.models.generate_content(

   model='gemini-3.5-flash',

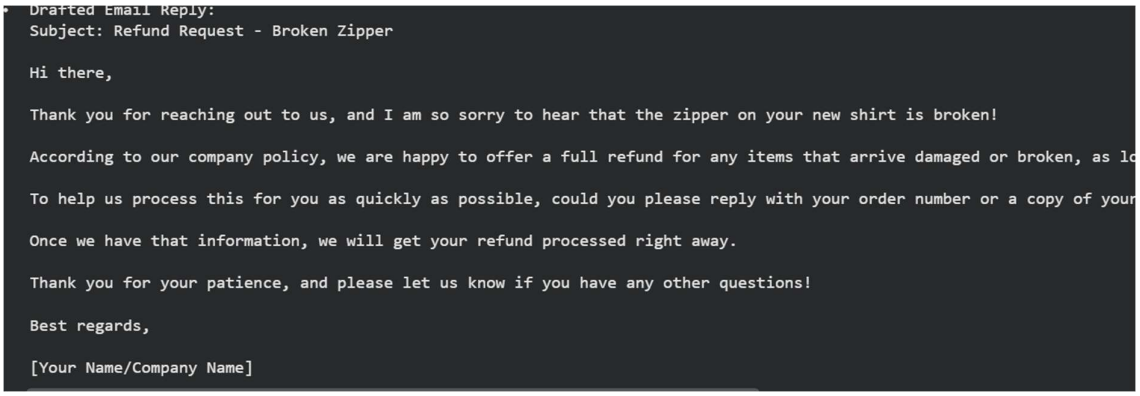
   contents=prompt

)

print("Drafted Email Reply:")

print(response.text)
```

## Output




## Conclusion

The practical demonstrated two applications of Generative AI. The first sub-part used the GPT-2 pre-trained model to generate text from a given prompt. The second sub-part demonstrated how Gemini can generate a context-aware customer-service response by using predefined rules and customer information. Thus, the practical provided an understanding of both **local text generation** and **prompt-based intelligent response generation**.

## Practical No. 6: Data Analysis and Visualization for AI-Based Insights

### Aim

To analyze a dataset using Python and perform data visualization to identify trends, patterns, and relationships that support **Predictive AI and Machine Learning workflows**.

### Theory

Artificial Intelligence systems depend on data to identify patterns and generate useful insights. Before applying Machine Learning or Predictive AI techniques, it is important to understand the available data through **Data Analysis and Visualization**.

**Exploratory Data Analysis (EDA)** is the process of examining and understanding a dataset using statistical methods and graphical representations.

In this practical, the **Superstore dataset** is analyzed using Python libraries:

- **Pandas** – used for loading, processing, grouping, and analyzing data.
- **Matplotlib** – used for creating graphs and charts.
- **Seaborn** – used for creating attractive statistical visualizations.

Three types of analysis are performed:

1. **Yearly Sales Trend** – identifies how sales change over different years.
2. **Sales by Category** – compares total sales among different product categories.
3. **Sales vs Profit** – examines the relationship between sales and profit.

These visualizations help understand the data before developing a **Predictive AI/ML model**. The analysis itself is not Generative AI; rather, it is a supporting stage in an AI workflow.

### AI Workflow

**Data → Data Analysis & Visualization → Machine Learning/Predictive AI → Predictions & Insights**

Generative AI can also be used at a later stage to summarize or explain the insights obtained from the analysis.

### Software and Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- Matplotlib
- Seaborn
- Superstore Dataset

### General Algorithm

1. Start.
2. Import the required Python libraries.
3. Load the Superstore dataset using Pandas.
4. Convert the **Order Date** column into datetime format.
5. Group the sales data according to the year.
6. Plot the yearly sales trend using a line chart.
7. Calculate and visualize total sales for each category using a bar chart.
8. Plot Sales against Profit using a scatter plot.
9. Analyze the graphs to identify trends and relationships.
10. Display the completion message.
11. Stop.

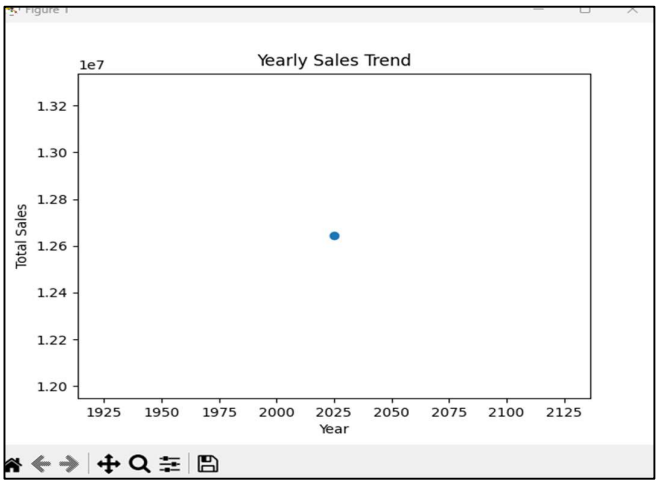
**Python Program**

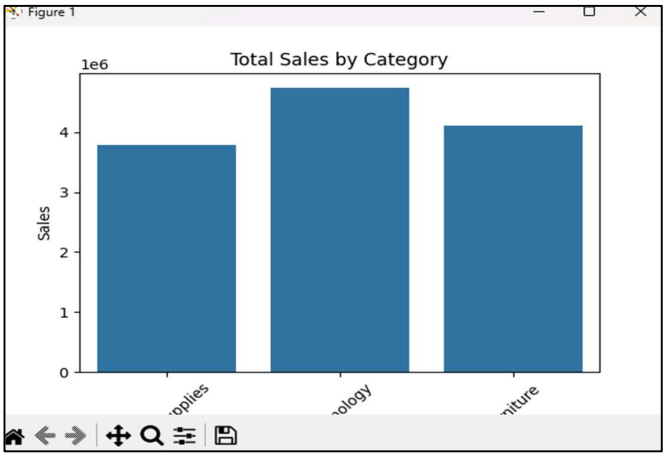
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

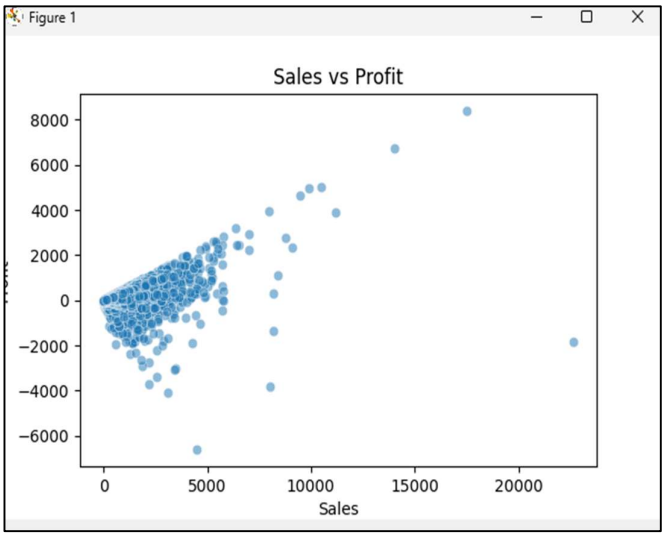
# Load dataset
df = pd.read_csv("/content/Sample - Superstore.csv", encoding='latin1')
# Convert Order Date to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])
# Sales trend over time
df_grouped = df.groupby(df['Order Date'].dt.year)['Sales'].sum()
df_grouped.plot(kind='line', marker='o', title='Yearly Sales Trend')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.show()
# Sales by Category
plt.figure(figsize=(6, 4))
sns.barplot(x=df['Category'], y=df['Sales'], estimator=sum, ci=None)
plt.title('Total Sales by Category')
```

```
plt.xticks(rotation=45)
plt.show()
# Profit vs. Sales scatter plot
plt.figure(figsize=(6, 4))
sns.scatterplot(x=df['Sales'], y=df['Profit'], alpha=0.5)
plt.title('Sales vs Profit')
plt.xlabel('Sales')
plt.ylabel('Profit')
plt.show()
```

**Sample Output**






**Interpretation**

The generated visualizations provide useful insights into the dataset:

- The **Yearly Sales Trend** helps identify whether sales are increasing or decreasing over time.
- The **Sales by Category** graph helps identify which product category contributes the most to total sales.
- The **Sales vs Profit** scatter plot helps understand the relationship between sales and profit.
- These observations can be useful for making business decisions and can also serve as a foundation for developing **Predictive AI and Machine Learning models**.

**Conclusion**

The practical successfully demonstrates **Data Analysis and Visualization using Python**. Pandas was used for data processing, while Matplotlib and Seaborn were used to visualize sales trends, category-wise sales, and the relationship between sales and profit.

## Practical No. 7: Dimensionality Reduction using PCA

### Aim

To implement **Principal Component Analysis (PCA)** using Python and reduce the dimensions of the Wine dataset for visualization.

### Theory

**PCA (Principal Component Analysis)** is a dimensionality reduction technique used in Machine Learning. It converts a dataset with many features into a smaller number of **principal components** while retaining important information.

In this practical, the Wine dataset containing **13 features** is standardized and reduced to **2 principal components**. The reduced data is then visualized using a scatter plot.

### Software and Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- Matplotlib
- Scikit-learn

### Algorithm

1. Import the required libraries.
2. Load the Wine dataset.
3. Create a DataFrame from the dataset.
4. Standardize the features.
5. Apply PCA with 2 components.
6. Plot the two principal components.
7. Display the graph.
8. Stop.

### Python Program

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

from sklearn.datasets import load_wine
```

```
# Load dataset
data = load_wine()

# Create DataFrame
df = pd.DataFrame(
    data.data,
    columns=data.feature_names
)
# Standardize data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)

# Apply PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(df_scaled)

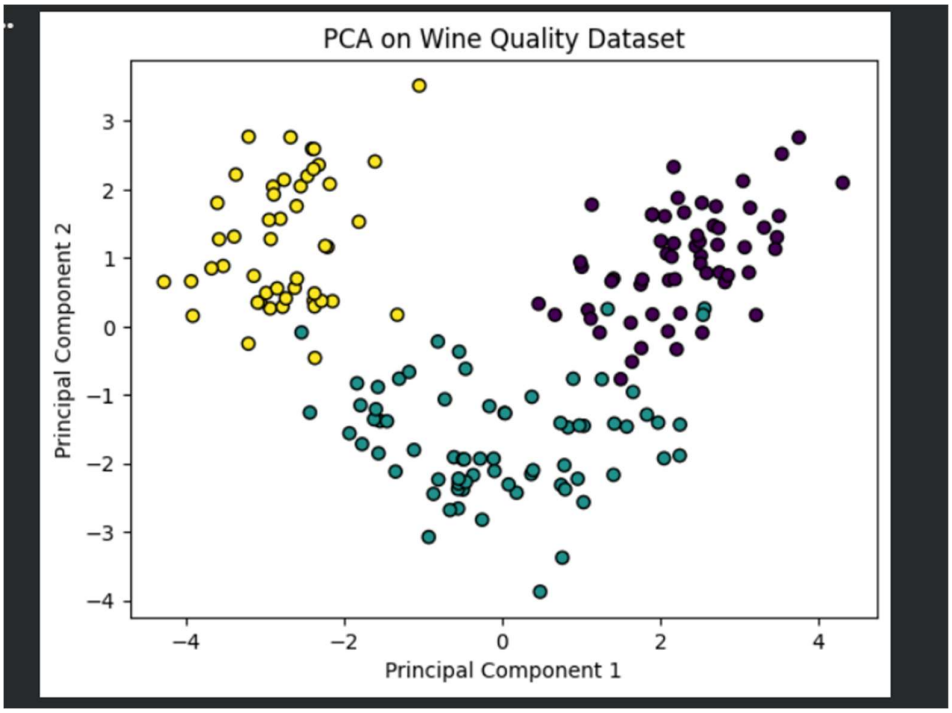
# Display reduced dimensions
print("Original Features:", df.shape[1])
print("Reduced Features:", pca_data.shape[1])

# Visualize PCA results
plt.scatter(
    pca_data[:, 0],
    pca_data[:, 1],
    c=data.target,
    cmap="viridis",
    edgecolors="k"
)

plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
plt.title("PCA on Wine Dataset")
plt.show()
```

**Sample Output**



## Practical No. 8: Statistical Analysis and Hypothesis Testing for AI

### Aim

To apply statistical techniques in Python for analyzing data and understanding how **ANOVA and hypothesis testing** support data-driven and AI-based decision-making.

### Objectives

1. Perform **One-Way ANOVA** to compare multiple groups and apply a post-hoc test.
2. Perform **One-Sample T-Test** to compare a sample mean with a hypothesized population mean.
3. Perform **Two-Sample T-Test** to compare the means of two independent groups.
4. Perform **Paired T-Test** to compare before-and-after measurements.
5. Formulate hypotheses and interpret the test results.

### Theory

**Hypothesis Testing** is a statistical method used to determine whether there is sufficient evidence to support a claim about data.

**ANOVA (Analysis of Variance)** is used to compare the means of three or more groups. If ANOVA shows a significant difference

**T-Tests** are used to compare means. One-sample, two-sample, and paired t-tests are used for different types of comparisons.

In **AI and Machine Learning**, statistical testing helps in **data analysis, feature evaluation, model evaluation, and evidence-based decision-making**.

### Part A: One-Way ANOVA

### Problem Statement

Consider the exam scores of students from three different classes. Perform a One-Way ANOVA test to determine whether there is a significant difference in the mean exam scores among the classes. If significant, perform a Tukey HSD post-hoc test.

**Class A:** [85, 90, 88, 82, 87]
**Class B:** [76, 78, 80, 81, 75]
**Class C:** [92, 88, 94, 89, 90]

### Hypothesis

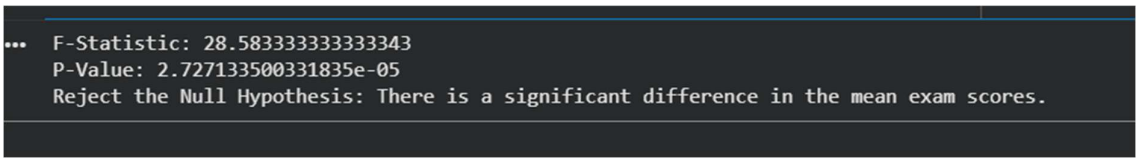
- **$H_0$ (Null Hypothesis):** There is no significant difference in the mean scores of the three classes.
- **$H_1$ (Alternative Hypothesis):** There is a significant difference in the mean scores of at least one class.

**Python Program**

```
import numpy as np
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd
# Data: exam scores for each class
class_A = [85, 90, 88, 82, 87]
class_B = [76, 78, 80, 81, 75]
class_C = [92, 88, 94, 89, 90]
# Perform One-Way ANOVA
f_statistic, p_value = stats.f_oneway(
    class_A, class_B, class_C
)
# Display results
print("F-Statistic:", f_statistic)
print("P-Value:", p_value)


# Draw conclusion
alpha = 0.05
if p_value < alpha:
    print("Reject the Null Hypothesis: There is a significant difference in the mean exam scores.")
else:
    print("Fail to Reject the Null Hypothesis: There is no significant difference in the mean exam scores.")
```

**Sample Output**



**Interpretation**

Since the p-value is less than 0.05, the null hypothesis is rejected. There is a significant difference between the mean scores of the classes. The Tukey test identifies which specific groups differ.

**Part B: One-Sample T-Test**

**Problem Statement**

Consider the following student scores and compare their mean with a hypothesized population mean of **70** using a One-Sample T-Test.

**Student Scores:** [72, 88, 64, 74, 67, 79, 85, 75, 89, 77]

**Hypothesis**

- **$H_0$:** The sample mean is equal to 70.
- **$H_1$:** The sample mean is significantly different from 70.

**Python Program**

```
import scipy.stats as stats

import numpy as np

# Given student scores

student_scores = [72, 88, 64, 74, 67, 79, 85, 75, 89, 77]

# Hypothesized population mean

hypothesized_mean = 70

# Perform one-sample t-test

t_statistic, p_value = stats.ttest_1samp(

    student_scores,

    hypothesized_mean

)

# Output the results

print(f"T-statistic: {t_statistic}")

print(f"P-value: {p_value}")

# Interpretation

alpha = 0.05

if p_value < alpha:

    print("Reject the null hypothesis: The sample mean is significantly different from 70.")
```

else:

print("Fail to reject the null hypothesis: The sample mean is not significantly different from 70.")

**Sample Output**

```
T-statistic: 2.6249999999999996
P-value: 0.027583862030934297
Reject the null hypothesis: The sample mean is significantly different from 70.
```

**Interpretation**

If the p-value is less than 0.05, the null hypothesis is rejected, indicating that the sample mean is significantly different from the hypothesized mean of 70.

**Part C: Two-Sample T-Test**

**Problem Statement**

A company wants to compare the time taken to complete a task between two groups of employees.

**Group I – Experience (0–1 year):**

[85, 95, 100, 80, 90, 97, 104, 95, 88, 92, 94, 99]

**Group II – Experience (1–2 years):**

[83, 85, 96, 92, 100, 104, 94, 95, 88, 90, 93, 94]

Perform a Two-Sample T-Test.

**Hypothesis**

- **$H_0$:** There is no significant difference in the mean task completion times.
- **$H_1$:** There is a significant difference in the mean task completion times.

**Python Program**

```
import scipy.stats as stats

import numpy as np

# Data for both groups

group1 = [85, 95, 100, 80, 90, 97, 104, 95, 88, 92, 94, 99]

group2 = [83, 85, 96, 92, 100, 104, 94, 95, 88, 90, 93, 94]

# Perform Two-Sample T-Test

t_stat, p_value = stats.ttest_ind(group1, group2)

# Display the result

print(f"T-statistic: {t_stat}")
```

```
print(f"P-value: {p_value}")


# Hypothesis testing conclusion
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference in the mean times.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in the mean times.")
```

**Sample Output**

```
T-statistic: 0.16119885358127883
P-value: 0.8734079490313532
Fail to reject the null hypothesis: There is no significant difference in the mean times.
```

**Interpretation**

If the p-value is greater than 0.05, the null hypothesis is not rejected. Therefore, there is no statistically significant difference between the mean task completion times of the two groups.

**Conclusion**

ANOVA and different types of T-Tests were successfully performed using Python. These statistical techniques help in **validating patterns, comparing groups, and supporting data-driven decisions in AI and Machine Learning applications**.